SmartUSBHub Python SDK

API 使用说明书

适用于: SmartUSBHub 产品系列（SDK v1.3.1）

正式版 · 文档更新：2026年11月12日

www.mixedsignallab.com

目录

简介 2

产品矩阵 2

产品概述 2

连接说明 2

CMD 控制口 3

快速上手 3

方法1：直接安装发布包 3

方法2：引用当前源码库 3

运行例程 3

集成到项目 3

用户接口 4

连接管理 4

通道控制 5

测量与监控 7

告警与保护 9

默认状态 11

设备配置 13

设备信息与发现 16

事件回调 18

文档导航：快速上手 · 通信协议 · 产品文档 · 示例说明

文档版本：v1.3.1 · 更新日期：2026年11月12日。

SDK 1.3.1 修复旧式 V3 测量数据和连续采集控制确认的兼容问题，公开 API 与协议 API 版本保持不变，详情见更新日志。

本文档适用于 SDK 支持的 SmartUSBHub 型号。使用前请确认设备型号，并选择对应的产品文档。

简介

SmartUSBHub Python 库用于在脚本、测试系统和自动化流程中控制 SmartUSBHub 设备。如果想直接用软件控制，请使用随发行包提供的配套控制软件，下载地址。

SmartUSBHub 不同型号的端口数量、供电能力、测量能力和专用功能可能不同，使用前请根据产品型号选择对应文档。

产品矩阵

产品名称	型号 / 订购代码	通道数量	USB 规格	技术规格书	使用指南
SmartUSBHub Pro 4CH USB2.0	HBP_USB2_4CH, HBP_USB2_4CH_PSU	4	USB 2.0 High-Speed	技术规格书	使用指南
SmartUSBHub Pro 7CH USB2.0	HBP_USB2_7CH, HBP_USB2_7CH_ADV	7	USB 2.0 High-Speed	技术规格书	使用指南
SmartUSBHub Pro 4CH USB3.0	HBP_USB3_4CH_ADV	4	USB 3.0 Super Speed	—	—

如使用其它受支持的订购代码，请查阅随该设备提供的型号专用资料。

产品概述

SmartUSBHub 是一系列可编程 USB 集线器，支持按通道控制电源；部分型号支持数据控制，部分型号支持电压/电流检测，适用于开发、自动化测试和设备管理。

每个下游端口可独立控制电源（VBUS）和数据连接：USB 2.0 型号控制 D+ / D−，支持 USB3.0 的型号还可独立控制 SuperSpeed 数据。无需手动插拔即可远程控制 USB 设备的连接与断开，用于自动化测试、远程重启和热插拔模拟。支持电压/电流检测的型号可实时采集各通道电压和电流，用于功耗分析与供电状态监测，测试中出现异常压降或电流过载时也能及时发现并定位。

每个端口可设置上电默认状态与断电状态记忆；支持普通模式（多通道同时控制）与互锁模式（仅允许操作一个通道）。支持集中控制多台设备的级联拓扑，每台设备可配置设备地址。控制口采用 USB CDC，免驱动，便于把端口控制集成到自动化测试平台、Python 脚本或其他工具中。

连接说明



提示：使用 USB 数据线连接设备的 DATA 口与 USB 主机

DATA 口是 USB 数据的上行口：下游 USB 设备经它连接主机，操作系统会识别到通用 USB Hub。

当同一台主机既要控制 SmartUSBHub，又要与下游 USB 设备通信时，DATA 与 CMD 都必须连接。

CMD 控制口

使用 USB 数据线连接设备的 CMD 控制口与控制计算机。CMD 口采用 USB CDC，免驱动，接入后系统枚举为标准串口：Windows 为 COMx，Linux 为 /dev/ttyACMx，macOS 为 /dev/cu.usbmodemx。

CMD 口的 USB 描述符为 VID 0x1A86、PID 0xFE0C，iSerialNumber 为设备序列号。
scan_available_ports() 按 VID/PID 过滤控制口；同一主机连接多台设备时，可通过序列号区分各设备。

快速上手

任选一种使用方式。

方法1：直接安装发布包

```
pip install smartusbhub
```

方法2：引用当前源码库

```
cd smartusbhub
python -m venv venv
source ./venv/bin/activate
python -m pip install -r requirements.txt
python -m pip install -e .
```

库目录结构如下：

```
.
├── README.md          # 文档
├── docs               # 产品指南与通信协议
├── examples           # 例程
├── requirements.txt   # SDK 核心依赖
└── smartusbhub.py     # 功能源码
```

运行例程

smartusbhub 库的示例存放在 examples 目录中，按顺序运行即可最快跑通设备：

- quick_start.py：打印完整设备报告（身份信息、能力、运行状态），随后演示电源控制：单通道、全通道同时开关、互锁模式、设备地址读写
- power_measurement_oneshot.py：在支持测量的型号上通过定长协议请求/响应持续读取各通道电压与电流
- multi_device_channel_control.py：发现并控制多台设备

详细说明见示例说明。

在包根目录下运行示例程序，例如：

```
python examples/quick_start.py
python examples/power_measurement_oneshot.py
```

集成到项目

通过导入 smartusbhub 库即可集成到你的项目之中。

SDK 采用 Apache-2.0 许可证。集成或再分发时，请遵循 Apache-2.0 第 4 条，以可读形式保留许可证、版权声明及 NOTICE 中的归属声明。

1. 导入 smartusbhub 库到你的工程：

```
from smartusbhub import SmartUSBHub
```

2. 初始化 SmartUSBHub 实例。自动扫描连接设备：

```
hub = SmartUSBHub.scan_and_connect()
```

或指定串口号连接（串口路径示例："/dev/cu.usbmodem132301"）：

```
hub = SmartUSBHub("/dev/cu.usbmodem132301")
```

用户接口

连接管理

scan_and_connect(exclude_ports=None, device_address=None)

描述：扫描匹配的 USB CDC 串口并连接第一台可用设备。

参数： exclude_ports（list[str]）：要跳过的串口集合；默认跳过已被连接的串口。 device_address（int）：只连接上报该地址的设备。设备地址默认为零，多台设备可能共用同一地址，配置前不具备唯一性，优先按串口选择。

返回值：SmartUSBHub 或 None：连接成功的实例；没有可用设备时返回 None。

示例：

```
hub = SmartUSBHub.scan_and_connect()
```

scan_available_ports()

描述：返回 USB VID/PID 与 SmartUSBHub 匹配的串口路径。

返回值：list[str]。

示例：

```
ports = SmartUSBHub.scan_available_ports()
```

auto_connect(exclude_ports=None, feature_filter=None)

描述：依次尝试候选串口，自动跳过被占用或连接失败的端口。

参数： exclude_ports（list[str]）：要跳过的串口集合；默认跳过已被连接的串口。 feature_filter（str）：只连接支持该功能的设备，可选 adc、usb2_data_switch、usb3_data_switch 或 ilim_switch。

返回值：SmartUSBHub 或 None。

示例：

```
hub = SmartUSBHub.auto_connect(feature_filter="adc")
```

SmartUSBHub(port)

描述：打开指定的 SmartUSBHub 串口。

参数： port（str）：设备串口路径，例如 "/dev/cu.usbmodem132301"。

示例：

```
hub = SmartUSBHub("/dev/cu.usbmodem132301")
```

disconnect()

描述：停止接收线程、关闭串口并释放进程锁。可以安全地重复调用。

示例：

```
hub.disconnect()
```

is_connected()

描述：返回串口当前是否处于打开状态。

返回值：bool。

close()

描述：disconnect() 的别名。

返回值：None。

示例：也可以用 with 语句管理连接，with 代码块结束时自动断开设备：

```
with SmartUSBHub("/dev/cu.usbmodem132301") as hub:
    hub.set_channel_power(1, state=1)
```

通道控制

set_channel_power(*channels, state)

描述：设置指定通道的电源状态。

参数： *channels（int）：要控制的通道。 state（int）：1 开启电源，0 关闭电源。

返回值：bool：如果命令设置成功返回 True，否则返回 False。

示例：

```
hub.set_channel_power(1, 2, state=1)
```

get_channel_power_status(*channels)

描述：查询指定通道的电源状态。

参数： *channels（int）：要查询的通道，可变参数形式，可用通道可通过 hub.get_channels() 获取。

返回值：dict 或 int 或 None：如果查询多个通道，返回包含通道状态的字典；如果查询单个通道，返回该通道的状态；若超时则返回 None。

示例：

```
status = hub.get_channel_power_status(1, 2)
```

set_channel_usb2_dataline(*channels, state)

描述：设置指定通道的USB数据开关状态。

参数：*channels（int）：要更新的通道，可变参数形式，可用通道可通过 hub.get_channels() 获取。 state（int）：1 表示连通 D+ / D−，0 表示断开 D+ / D−。

返回值：bool：如果命令设置成功返回 True，否则返回 False。

示例：连通通道 1 的数据信号。

```
hub.set_channel_usb2_dataline(1,state=1)
```

get_channel_usb2_dataline_status(*channels)

描述：查询指定通道的USB数据开关状态。

参数：*channels（int）：要查询的通道，可变参数形式，可用通道可通过 hub.get_channels() 获取。

返回值：dict 或 None：包含通道状态的字典，若超时则返回 None。

示例：获取通道 1、2 的 USB 2.0 数据连接状态。

```
status = hub.get_channel_usb2_dataline_status(1, 2)
```

set_channel_usb3_dataline(*channels, state)



提示
注意: 此 API 仅适用于声明 USB3.0 数据开关能力的型号（如 HBP_USB3_4CH_ADV）。不支持的设备会抛出 FeatureNotSupportedError。

描述：设置指定通道的 USB 3.0 数据开关状态。

参数：*channels（int）：要更新的通道，可变参数形式，可用通道可通过 hub.get_channels() 获取。 state（int）：1 表示连通，0 表示断开。

返回值：bool：设备确认返回 True，超时返回 False。

示例：

```
hub.set_channel_usb3_dataline(1, state=1)
```

get_channel_usb3_dataline_status(*channels)

描述：查询指定通道的 USB 3.0 数据开关状态；未指定通道时查询全部有效通道。

参数：*channels（int）：要查询的通道。

返回值：dict 或 None：{通道号: 状态}；超时返回 None。

示例：

```
status = hub.get_channel_usb3_dataline_status(1, 2)
```

set_channel_power_interlock(channel)

描述：在互锁模式下选择唯一供电通道，或关闭所有通道。关闭通道电源时，该通道的数据连接也会随之断开。

注意：互锁模式下，普通电源控制指令 set_channel_power() 无效，必须使用 set_channel_power_interlock() 选择供电通道。互锁指令本身控制电源；如需控制所选通道的 USB 2.0 数据连接，请使用 set_channel_usb2_dataline()。

参数： channel（int 或 None）：要设置的通道；None 表示关闭所有通道。

返回值：bool：如果命令设置成功返回 True，否则返回 False。

示例：

```
hub.set_channel_power_interlock(1)
```

set_upstream_host(host)



提示
注意: 此 API 仅适用于声明 upstream_host_switch 能力（支持双主机切换）的型号。不支持的设备会抛出 FeatureNotSupportedError。

描述：选择设备连接的上游主机。选择保存到设备，断电后保持。

参数： host（int）：上游主机编号，1 或 2。

返回值：bool：收到匹配确认返回 True；被设备拒绝或超时返回 False。

示例：切换到主机 2。

```
hub.set_upstream_host(2)
```

get_upstream_host()

描述：查询设备当前使用的上游主机。

返回值：int 或 None：0（未知）、1（主机 1）或 2（主机 2）；被拒绝或通信失败返回 None。

示例：

```
host = hub.get_upstream_host()
```

测量与监控

get_channel_voltage(channel)



提示
注意: 此API仅适用于 带有电压、电流检测的 型号。

描述：查询单个通道的电压。

参数： channel（int）：要查询的通道。

返回值：int 或 None：通道的电压值(mV)，若超时则返回 None。

示例：获取通道 1 的电压值。

```
voltage = hub.get_channel_voltage(1)
```

get_channel_current(channel)

> 提示
> 注意: 此API仅适用于 带有电压、电流检测的 型号。

描述：查询单个通道的电流。

参数： channel（int）：要查询的通道。

返回值：int 或 None：通道的电流值(mA)，若超时则返回 None。

示例：获取通道 1 的电流值。

```
current = hub.get_channel_current(1)
```

get_channel_measurements(*channels)

> 提示
> 注意: 此 API 仅适用于 带有电压、电流检测的 型号。

描述：一次请求读取一个或多个通道的电压、电流测量值。未指定通道时，读取当前产品的所有有效通道。

参数： *channels（int）：要查询的通道，可变参数形式，可用通道可通过 hub.get_channels() 获取。

返回值：dict 或 None：返回 {通道号: {"voltage": mV, "current": mA, "fresh": bool, "stale": bool, "valid": bool}}，若超时或无有效数据则返回 None。

示例：

```
measurements = hub.get_channel_measurements(1, 2)
all_measurements = hub.get_channel_measurements(*hub.get_channels())
```

set_channel_measurement_stream(*channels, enabled=True, wait_ack=True)

> 提示
> 注意: 此 API 仅适用于支持连续测量数据输出的型号。

描述：开启或关闭指定通道的连续测量数据输出。未指定通道时，默认作用于当前产品的所有有效通道。

参数： *channels（int）：要输出测量数据的通道。 enabled（bool）：True 开启，False 关闭。 wait_ack（bool）：是否等待设备应答。

返回值：bool：成功返回 True，超时返回 False。

示例：

```
hub.set_channel_measurement_stream(*hub.get_channels(), enabled=True)
```

get_stream_channel_measurements(*channels, timeout=None, wait_new_sample=True)

描述：等待下一帧连续测量数据并返回指定通道的测量值。调用前需要先开启连续测量数据输出。

参数： *channels（int）：要读取的通道。 timeout（float）：等待超时时间，单位为秒；不传则使用默认通信超时。 wait_new_sample（bool）：True 时等待新的采样点，False 时接受下一帧数据。

返回值：dict 或 None：返回包含 voltage、current、sample_tick、sample_period_ms 等字段的字典，超时返回 None。

批量采集按顺序逐条消费，不会只保留批次最后一条。sample_time_us 是 MCU 发出 REFRESH 请求的时间（按 32 位微秒回绕），不是精确的 ADC 转换时刻；capture_statistics 与 measurement_queue_drops 分别报告设备侧和主机侧的丢样情况。型号相关的采样档位与滤波特性见对应产品文档。

示例：

```
data = hub.get_stream_channel_measurements(1, 2, timeout=1.0)
```

get_cached_measurements(*channels)

描述：直接读取后台接收线程缓存的最新测量值，不阻塞等待新数据。通常与连续测量数据输出配合使用。

参数： *channels（int）：要读取的通道。

返回值：dict 或 None：返回最新缓存测量值；如果还没有收到测量数据，则返回 None。

示例：

```
latest = hub.get_cached_measurements(*hub.get_channels())
```

get_measurement_sample_rate() 与 set_measurement_sample_rate(rate)

描述：读取或设置设备声明的测量采样率（单位 Sa/s）。修改期间会暂停并恢复正在进行的采集；设置未声明的采样率会抛出 FeatureNotSupportedError。

参数： rate（int）：目标采样率，单位 Sa/s，必须是设备声明的档位。

返回值：读取时返回 int 或 None；设置时返回 bool。

相关：supported_sample_rates() 返回设备声明的采样率元组；supports_measurement_capture() 返回设备是否支持批量采集。

示例：

```
rates = hub.supported_sample_rates()
if 100 in rates:
    hub.set_measurement_sample_rate(100)
```

get_measurement_filter() 与 set_measurement_filter(mode)

描述：读取或选择测量硬件滤波模式。设备未声明可配置滤波时抛出 FeatureNotSupportedError。设置只保存在 RAM 中，重启后默认为瞬时值；切换会短暂停流并清空旧批次。

参数： mode（int）：0 为瞬时值，1 为硬件内置的八次转换滑动平均，输出采样率不变。

返回值：读取时返回 int（0/1）或 None；设置时返回 bool。

告警与保护

get_channel_oc_status()



提示
注意：HBP_USB2_7CH 与 HBP_USB2_7CH_ADV 都支持各端口过流状态。其他型号是否支持取决于硬件和固件。

描述：查询每个通道的过流实时状态和锁存状态。

返回值：dict 或 None：返回 {通道号: {"active": bool, "latch": bool}}。active 表示当前过流状态，latch 表示锁存过流事件。若超时则返回 None。

示例：

```
oc_status = hub.get_channel_oc_status()
```

clear_channel_oc_latch(*channels)

描述：清除指定通道的过流锁存状态。未指定通道时，清除所有通道的锁存状态。

参数： *channels（int）：要清除锁存状态的通道。

返回值：bool：成功返回 True，超时返回 False。

示例：

```
hub.clear_channel_oc_latch(1, 2)
hub.clear_channel_oc_latch()
```

> **提示**
> 注意: 此组 API 仅适用于设备声明 alerts 能力的型号（如 HBP_USB3_4CH_ADV）。不支持的设备会抛出 FeatureNotSupportedError。

阈值规则支持 overcurrent（过流）、undercurrent（欠流）、overvoltage（过压）、undervoltage（欠压）、overpower（过功率）五种类型，阈值单位分别为 mA、mA、mV、mV、mW。比较器阈值按设备声明量化，布局与限制见协议说明。

规则默认关闭，保存在 RAM 中；set_alert_config() 成功返回前会把规则写入应用 Flash（相同配置不会重复擦写）。action 可选 notify（仅上报）或 power_off_lock（断电并锁定：断开电源和数据连接，并阻断所有上电路径，直到显式解锁）。

set_alert_config(channel, kind, threshold, enabled=True, samples=1, action="notify")

描述：设置一条阈值规则。

参数： channel（int）：规则所属通道。 kind（str）：告警类型，取值见上方五种类型。 threshold（int）：阈值，单位随类型为 mA、mV 或 mW。 enabled（bool）：是否启用该规则。 samples（int）：比较器连续命中次数，可选 1/4/8/16。 action（str）：notify（仅上报）或 power_off_lock（断电并锁定）。

返回值：dict 或 None：设备实际应用的规则字典，失败返回 None。

get_alert_config(channel, kind) 与 get_alert_defaults(channel, kind)

描述：分别读取当前生效的规则和 MCU 出厂默认规则，不改变现有配置。

参数： channel（int）：规则所属通道。 kind（str）：告警类型，取值见上方五种类型。

返回值：dict 或 None。

get_alert_status()

描述：读取缓存的告警状态，包含 ready、io_fault、事件序号、丢事件数，以及各通道的 active（当前告警）、history（告警历史）、trip（保护锁定）、power（供电状态）。不消耗芯片的读清除寄存器。

返回值：dict 或 None。

subscribe_alert_events(enabled=True)

描述：订阅设备主动上报的告警事件。需先通过 register_callback() 注册 CMD_ALERT_EVENT 回调，回调在接收线程中执行。

参数： enabled（bool）：True 订阅，False 取消订阅。

返回值：bool。

clear_alert_latch(channel_mask) 与 reset_protection_trip(channel_mask)

描述：清除告警历史或解锁保护锁定。clear_alert_latch() 仅清除告警历史，不解锁保护；reset_protection_trip() 在监测读数恢复健康后解锁已断电端口——它不会自动上电，重新上电是显式操作，且负载故障未排除时会再次触发保护。

参数： channel_mask（int）：通道位掩码，位 0 对应通道 1。

返回值：bool。

示例：

```
hub.set_alert_config(1, "overcurrent", 2000, samples=8, action="power_off_lock")
status = hub.get_alert_status()
# 故障处理后，端口保持断电，告警解除且监测正常时：
hub.clear_alert_latch(0x01)     # 仅清历史
hub.reset_protection_trip(0x01) # 仅解锁，不上电
```

默认状态

set_default_power_status(*channels, enable, status=None)

描述：设置指定通道的上电默认电源状态。

参数： *channels（int）：要设置的通道，可变参数形式，可用通道可通过 hub.get_channels() 获取。enable（int 或 bool）：1/True 启用默认状态，0/False 禁用。 status（int 或 bool，可选）：1/True 默认打开电源，0/False 默认关闭；省略时为关闭。

返回值：bool：如果命令设置成功返回 True，否则返回 False。

示例：通道1、2、3、4上电默认打开。

```
hub.set_default_power_status(1, 2, 3, 4, enable=1, status=1)
```

示例：通道1、2、3、4上电不使用默认值。

```
hub.set_default_power_status(1, 2, 3, 4, enable=0)
```

get_default_power_status(*channels)

描述：查询一个或多个通道电源的默认上电状态。

参数： *channels（int）：要查询的通道，可变参数形式，可用通道可通过 hub.get_channels() 获取。

返回值：dict 或 None：{通道号: {"enabled": 是否启用, "value": 状态}}，其中 enabled 为 0（禁用）或 1（启用），value 为 0（默认关闭）或 1（默认开启）。若超时则返回 None。

示例：查询尚未配置过默认状态的设备，获取通道 1、2、3、4 的电源上电默认状态：

```
hub.get_default_power_status(1, 2, 3, 4)
```

返回：

```
{1: {'enabled': 0, 'value': 0}, 2: {'enabled': 0, 'value': 0}, 3: {'enabled': 0, 'value': 0}, 4: {'enabled': 0, 'value': 0}}
```

enabled 为 0 表示该通道不使用上电默认值，此时 value 不生效；只有 enabled 为 1 时，value 才决定上电时电源打开还是关闭。

set_default_usb2_dataline_status(*channels, enable, status=None)

描述：设置指定通道的上电默认数据连接状态。

参数： *channels（int）：要设置的通道，可变参数形式，可用通道可通过 hub.get_channels() 获取。enable（int 或 bool）：1/True 启用默认状态，0/False 禁用。status（int 或 bool，可选）：1/True 默认连接数据，0/False 默认断开；省略时为断开。

返回值：bool：如果命令设置成功返回 True，否则返回 False。

示例：通道1、2、3、4上电默认数据连接。

```
hub.set_default_usb2_dataline_status(1,2,3,4,enable=1,status=1)
```

get_default_usb2_dataline_status(*channels)

描述：查询一个或多个通道的上电默认数据连接状态。

参数： *channels（int）：要查询的通道，可变参数形式，可用通道可通过 hub.get_channels() 获取。

返回值：dict 或 None：{通道号: {"enabled": 是否启用, "value": 状态}}，其中 enabled 为 0（禁用）或 1（启用），value 为 0（默认断开）或 1（默认连接）。若超时则返回 None。

示例：获取通道1、2、3、4的数据连接的上电默认状态。

```
hub.get_default_usb2_dataline_status(1,2,3,4)
```

返回：

```
{1: {'enabled': 0, 'value': 1}, 2: {'enabled': 0, 'value': 1}, 3: {'enabled': 0, 'value': 1}, 4: {'enabled': 0, 'value': 1}}
```

set_default_usb3_dataline_status(*channels, enable, status=None)



提示
注意: 此 API 仅适用于声明 device.default_usb3_data 能力的型号（如 HBP_USB3_4CH_ADV）。不支持的设备会抛出 FeatureNotSupportedError。

描述：设置指定通道 USB3.0 数据的上电默认状态，并保存到设备。设置保存成功后不立即改变当前数据连接，在设备上电恢复时生效。

参数： *channels（int）：要设置的通道，可变参数形式，可用通道可通过 hub.get_channels() 获取。enable（int 或 bool）：1/True 启用默认状态，0/False 禁用。status（int 或 bool，可选）：1/True 默认连接数据，0/False 默认断开；省略时为断开。

返回值：bool：固件保存并回读校验成功返回 True，否则返回 False。

注意：enable=0 时该端口恢复使用普通上电默认状态。上电恢复时 USB2.0 默认断开和电源关闭优先于 USB3.0 默认状态。

示例：通道1、2 上电默认断开 USB3.0 数据。

```
hub.set_default_usb3_dataline_status(1, 2, enable=1, status=0)
```

get_default_usb3_dataline_status(*channels)



提示
注意: 此 API 仅适用于声明 device.default_usb3_data 能力的型号（如 HBP_USB3_4CH_ADV）。不支持的设备会抛出 FeatureNotSupportedError。

描述：查询一个或多个通道的 USB3.0 数据上电默认状态。

参数： *channels（int）：要查询的通道，可变参数形式，省略时查询全部通道。

返回值：dict 或 None：{通道号: {"enabled": 是否启用, "value": 状态}}，含义与 USB2.0 默认状态相同。若超时则返回 None。

示例：获取通道1、2 的 USB3.0 数据上电默认状态。

```
hub.get_default_usb3_dataline_status(1, 2)
```

set_auto_restore(enable)

描述：启用或禁用断电状态恢复。

参数： enable（bool）：True 启用，False 禁用。

返回值：bool：如果命令设置成功返回 True，否则返回 False。

示例：启用断电状态恢复。

```
hub.set_auto_restore(True)
```

get_auto_restore_status()

描述：查询是否启用断电状态恢复。

返回值：int 或 None：1 表示启用，0 表示禁用，设备无响应时返回 None。

示例：查询断电状态恢复。

```
status = hub.get_auto_restore_status()
```

设备配置

set_button_control(enable)

描述：启用或禁用集线器的物理按钮（如有）。

参数： enable（bool）：True 启用按钮，False 禁用按钮。

返回值：bool：如果命令设置成功返回 True，否则返回 False。

示例：设置按钮为启用。

```
hub.set_button_control(True)
```

get_button_control_status()

描述：查询集线器的物理按钮是否启用（如有）。

返回值：int 或 None：1 表示启用，0 表示禁用，若无响应则返回 None。

示例：查询按钮是否启用。

```
status = hub.get_button_control_status()
```

set_device_address(address)

描述：设备地址用于在多台集线器连接的场景中，标识和区分各个集线器。

参数： address（int）：用户自定义地址，范围为 0x0000～0xFFFF。

返回值：bool：设备确认修改时返回 True，否则返回 False。

提示：创建 SmartUSBHub 实例时会自动获取连接设备的地址。多设备按地址选择前，必须先为每台设备配置唯一地址。

示例：设置设备地址为 0x0001。

```
hub.set_device_address(0x0001)
```

get_device_address()

描述：获取设备的地址。

返回值：int 或 None：设备地址，若无响应则返回 None。

示例：查询设备地址。

```
device_address = hub.get_device_address()
```

set_operate_mode(mode)

描述：设置设备的操作模式。

参数： mode（int）：操作模式（0 为普通模式，1 为互锁模式）。

返回值：bool：如果命令设置成功返回 True，否则返回 False。

注意：互锁模式下，电源只能通过 set_channel_power_interlock() 控制；普通电源指令无效。通道断电时数据连接会同步断开。

示例：设置设备为普通模式。

```
hub.set_operate_mode(0)
```

get_operate_mode()

描述：查询设备的当前操作模式。

返回值：int 或 None：当前操作模式，若无响应则返回 None。

示例：查询设备操作模式。

```
mode = hub.get_operate_mode()
```

factory_reset()

描述：将设备重置为出厂设置。

返回值：bool：如果命令设置成功返回 True，否则返回 False。

示例：

```
hub.factory_reset()
```

identify_device()

描述：让当前设备的状态指示灯快速闪烁，便于在多台设备中完成物理定位。

返回值：bool；设备确认命令时返回 True，否则返回 False。

set_device_alias(alias) 与 get_device_alias()

描述：设置或读取 UTF-8 设备别名。

参数： alias（str）：UTF-8 别名，最多 15 个字节；空字符串清除别名。

返回值：set_device_alias() 返回 bool；get_device_alias() 返回别名字符串，未设置或旧固件不支持时返回空字符串。

示例：

```
hub.set_device_alias("机架 A")
print(hub.get_device_alias())
```

set_channel_name(channel, name) 与 get_channel_name(channel)

描述：设置或读取单通道 UTF-8 显示名称。

参数： channel（int）：通道号。 name（str）：UTF-8 名称，最多 15 个字节；空字符串恢复默认 CHn。

返回值：set_channel_name() 返回 bool；get_channel_name() 返回已存名称或默认通道名。

get_channel_names(*channels)

描述：读取指定通道的显示名称。

参数： *channels（int）：要读取的通道；未传通道时读取全部有效通道。

返回值：dict[int, str]。

示例：

```
hub.set_channel_name(1, "被测设备")
names = hub.get_channel_names()
```

reboot_mcu()

描述：请求 MCU 重启。设备确认后很快断开，通常需要重新连接。

返回值：bool；设备确认命令时返回 True，否则返回 False。



提示：重启前必须停止下游设备的写入和固件操作

reset_hub()

描述：复位设备内的 USB 数据 Hub。下游 USB 设备会断开并可能重新枚举；MCU、控制串口和端口供电策略不受影响。

返回值：bool；命令执行成功返回 True，否则返回 False。



提示
注意: 此 API 仅适用于声明 device.reset_hub 能力的型号。不支持的设备会抛出 FeatureNotSupportedError。

设备信息与发现

get_channels()

描述：返回当前连接产品的所有有效通道编号，通道编号从 1 开始。

返回值：tuple：当前设备可用通道，例如 (1, 2, 3, 4) 或 (1, 2, 3, 4, 5, 6, 7)。

示例：

```
channels = hub.get_channels()
hub.set_channel_power(*channels, state=1)
```

get_identity()

描述：读取缓存的设备身份信息，包括串口路径、序列号、地址、别名、型号、通道数、硬件/固件版本、工作模式等。字段值为原始类型，可直接序列化为 JSON；与 get_device_info() 不同，缺失的值保持 None 而不是 "N/A"。

相关：能力、规格、Hub 身份等声明信息推荐使用 get_capability()；MCU 序列号、地址及版本使用 get_identity()；get_device_info() 保留为旧版汇总/重新查询接口，不再推荐作为普通读取入口。

返回值：dict：设备身份信息字典。

示例：

```
identity = hub.get_identity()
print(identity["model"], identity["serial"])
```

get_product_type()

描述：查询当前连接设备的产品类型 ID。

返回值：int 或 None：产品类型 ID，若无响应则返回 None。

示例：

```
product_type = hub.get_product_type()
```

get_capability(refresh=False)

描述：返回固件 JSON 能力声明的 Python 字典（独立副本），包含摘要及已声明的详情分组。默认读取连接时缓存；没有固件声明时返回 None，不按型号伪造字段。 顶层能力可能是布尔值，也可能是带 supported 的字典，以固件声明结构为准。 需要 JSON 文本时使用 json.dumps(caps, ensure_ascii=False)。

参数： refresh（bool）：True 时重新查询摘要及已声明的详情分组。

返回值：dict 或 None。

示例：

```
caps = hub.get_capability()
if caps:
    measurement = caps.get("measurement")
    supported = measurement is True or (
        isinstance(measurement, dict) and measurement.get("supported") is True
    )
    if supported:
        values = hub.get_channel_measurements()
```

get_product_name()

描述：查询当前连接设备的产品名称，例如 HBP_USB2_4CH 或 HBP_USB2_7CH_ADV。

返回值：str 或 None：产品名称，若无响应则返回 None。

示例：

```
product_name = hub.get_product_name()
```

get_max_channels()

描述：查询当前设备的最大通道数量。新代码通常应优先使用 get_channels() 获取实际可用通道列表。

返回值：int 或 None：最大通道数量；旧固件可能不支持该命令，此时返回 None。

示例：

```
max_channels = hub.get_max_channels()
channels = hub.get_channels()
```

get_serial_no()

描述：查询设备序列号。

返回值：str 或 None：设备序列号；如果设备未提供序列号，可能返回 "N/A"。

示例：

```
serial_no = hub.get_serial_no()
```

get_firmware_version()

描述：查询设备的固件版本。

返回值：int 或 None：固件版本，若无响应则返回 None。

示例：

```
firmware_version = hub.get_firmware_version()
```

get_firmware_version_major()

描述：返回缓存的固件主版本；必要时先查询设备。旧版固件按主版本 1 处理。

返回值：int 或 None。

get_firmware_version_string()

描述：把缓存的固件版本格式化为显示字符串，例如 V2.1。

返回值：str；无法读取时返回 "Unknown"。

get_hardware_version()

描述：查询设备的硬件版本。

返回值：int 或 None：硬件版本，若无响应则返回 None。

示例：

```
hardware_version = hub.get_hardware_version()
```

事件回调

register_callback(cmd, callback)

描述：为指定的命令注册一个用户回调函数。当设备返回该命令的应答或主动上报对应事件时，回调函数会被触发。

参数： cmd（int 或 str）：要注册回调的命令码；也可传 "disconnect" 注册主机侧异常断连事件，此时在端口清理后由接收线程调用无参数回调，主动调用 disconnect() 不触发。 callback（function）：命令的 ACK 被接收到时执行的回调函数。命令码回调接受两个参数——channel（int）为触发回调的通道编号，status（int）为通道的状态值；"disconnect" 回调无参数。传入 None 可取消注册，同一事件再次注册会替换旧回调。

返回值：无。

注意：可注册的命令码来自 SDK 导出的 CMD_* 常量：所有需要等待设备应答的命令，外加 CMD_ALERT_EVENT、CMD_DEBUG_LOG 两个主动事件；cmd="disconnect" 是主机侧事件，不使用命令码。传入未包含在内的命令码会记录警告日志，不会注册回调。回调函数的签名应与设备返回的数据结构匹配。普通控制和查询指令使用定长协议帧。变长协议用于高速或流式数据传输，例如测量数据流；普通应用代码不需要手动选择协议版本。

示例：设置按键回调，当按键按下时，产生回调。

```
from smartusbhub import CMD_GET_CHANNEL_POWER_STATUS

def button_press_callback(channel, status):
    print("Button press detected on channel", channel, "with power status", status)

hub.register_callback(CMD_GET_CHANNEL_POWER_STATUS, button_press_callback)
```